



Tines Connector — Credential Provisioning Guide

Version 0.2.0, 2026-08-12

Table of Contents

Overview.....	1
Which kind of API key to create	1
Why this connector cannot run a story.....	2
Generating the credential.....	2
Mapping values into Crogl.....	3
Verifying the setup.....	3

Overview

The Tines connector reads **stories** (Tines' unit of automation—what most SOC teams call a playbook) and the **actions** inside them from a Tines tenant over its `/api/v1` REST API. It authenticates with a **Tines API key**, sent as an `Authorization: Bearer` header.

Vendor documentation for everything in this guide:

- Authentication and authorization—the API key categories in [Which kind of API key to create](https://www.tines.com/stories/docs/api/authentication/): <https://www.tines.com/stories/docs/api/authentication/>
- API reference index (rate limits, endpoint list): <https://www.tines.com/stories/docs/api/>
- List stories endpoint—the call this connector's story listing makes: <https://www.tines.com/stories/docs/api/stories/list/>
- List actions endpoint—the call behind webhook-trigger discovery: <https://www.tines.com/stories/docs/api/stories/actions/list/>
- Self-hosted release notes—check your build here when an endpoint behaves differently than this guide describes: <https://www.tines.com/stories/docs/self-hosted/release-notes/>

These steps are performed by two people:

- A **Tines tenant administrator** (or a user with permission to mint API tokens) creates the token and confirms which teams it can see.
- Whoever administers your Crogl instance enters the resulting values when creating the connector.

This connector is **read-only**. It lists and describes stories and agents; it never runs one. The token therefore only needs **read** access, and specifically should **not** be granted the "send to story" permission—see [Why this connector cannot run a story](#).

Which kind of API key to create

Tines has **four categories** of API key, and this is the most consequential choice in this guide:

Category	What it can reach
Personal	Linked to your user account; has access to everything that user does. Tied to an individual, so it breaks when they leave.
Service	Linked to a separate service-account user. A tenant owner can grant it specific tenant permissions. Recommended.
Team	Role-based access to one specific team. Recommended when Crogl only needs one team's stories.
Tenant owner	Full Owner access to the entire tenant. Do not use for this connector.

Use a **Service** or **Team** key. A Team key is the tighter choice when the stories Crogl needs live in a single team; a Service key suits spanning several teams or attaching the credential to a service

account you already manage.

There is no separate read/write scope switch, and there is nothing to withhold for this connector: it only reads stories, so it needs no execution permission in any category.

NOTE

Tines is partitioned by **team**, and by **folder** within a team. A key sees only the stories its access covers, and a story outside that scope does **not** produce a permission error — see the 404 note under [Verifying the setup](#), because that is the most confusing failure mode in this integration.

Tenant owners can also restrict who may create API keys; if the **API keys** screen offers no create option, you need the **API_KEY_CREATE** permission, or ask an owner to mint the key.

Why this connector cannot run a story

Tines has two ways to run a story remotely: **Send-to-Story** (`POST /api/v1/stories/:id/events`) and a **webhook** (`POST /webhook/<path>/<secret>`), authenticated by the secret embedded in the URL).

This connector exposes **neither**. There is no subcommand that can run a story, so the key you create here does not need — and should not be given — any permission beyond reading stories.

IMPORTANT

A webhook agent's secret **is** the full authentication for that webhook — anyone holding `https://<your-tenant>/webhook/<path>/<secret>` can trigger that story. Those secrets are returned by the Tines agents API, so the connector strips every **secret** value out of the payload before it is displayed, cached, or written to the knowledge graph. If you need a webhook URL for some other purpose, get it from the Tines UI — do not ask Crogl to reveal one.

*The menu labels below are from Tines' published documentation. Steps marked **(verify on your tenant)** could not be confirmed against a live Tines console for this guide; the wording may differ slightly on your version.*

Self-hosted note: Tines renamed "agents" to "actions", and this connector reads webhook-trigger data from the documented `/api/v1/actions` endpoint, falling back to the older `/api/v1/agents` path automatically. Both are tried, so no configuration is needed either way. If webhook-trigger discovery misbehaves on your build, check your version against the [release notes](#) — the docs do not state which release introduced the `/api/v1/actions` path, which is why the connector probes rather than assuming.

Generating the credential

Per Tines' authentication documentation (**verify on your tenant — exact labels vary by version**):

1. Open **Settings**.
2. Select **API keys** under **Access & security**.
3. Create a new key, choosing the **Service** or **Team** category per the table above. For a Team key,

select the team whose stories Crogl should read.

4. Give it a recognizable name, e.g. `crogl-connector`.
5. **Copy the key value immediately** — it is shown only once. Store it in your secret manager; you will paste it into Crogl in the next section.

Tines does not publish a key expiry or rotation procedure, so treat the key as long-lived and rotate it deliberately: create the replacement, update the connector, then delete the old key.

TIP Do not confuse this with **Credentials** under team settings (+ **New Credential** → **Tines**). That screen creates a credential for use **inside** Tines stories, which is the opposite direction of travel from what this connector needs.

Also note your Tines **tenant host** (for example `acme.tines.com`). You need the hostname only — no `https://` scheme and no `/api/v1` path.

Mapping values into Crogl

One row per `connector.json` credential field:

Crogl field	Where it came from	Notes
<code>host</code>	Your Tines tenant URL	Bare FQDN, e.g. <code>acme.tines.com</code> . No <code>https://</code> and no <code>/api/v1</code> — Crogl adds the scheme and the connector adds the API path. Self-hosted tenants use their own FQDN here.
<code>api_key</code>	The API key copied above	Secret. Sent as <code>Authorization: Bearer <key></code> . Tines also accepts the same key in an <code>x-user-token</code> header; this connector binds the Bearer form.
<code>skip_tls_verification</code>	n/a (operator choice)	Leave off for Tines Cloud, which presents a certificate from a public CA. Turn on only for a self-hosted tenant behind a private/internal CA, and only if you accept skipping certificate verification.

Verifying the setup

After creating the connector instance in Crogl, ask it to list your stories. Interpret the result:

- **Rows returned** — success. The footer also reports how many stories match tenant-wide, so you can tell at a glance whether the token sees everything you expected.
- **No query results (0 rows)** — the call succeeded and matched nothing. Two common causes: the token's user is not a member of the team holding the stories, or you filtered by a tag that doesn't exist. Tag filters are **exact matches**, not substrings, and untagged stories can't be tag-filtered at all.

- **401 Unauthorized** — the key is wrong or was revoked. Reissue it under **Settings** → **API keys** and re-enter `api_key`. (Crogl does not validate the key when the connector is created; a bad key first appears here, on the first real query. Tines does not document key expiry, so don't read this as a lapsed TTL.)
- **404 Not Found** — **this is the one to read carefully**. Tines documents that an **underprivileged API key** hitting a resource it lacks permission for returns **404: Not Found**, not **401** or **403**. So a 404 means one of three different things:
 1. **The key lacks permission**. The story exists; your key's category or team scope doesn't cover it. This is why a permissions problem looks identical to a missing story.
 2. **The host value is wrong** — a stray `https://`, an `/api/v1` path baked in, or a misspelled tenant subdomain. This 404s **every** call, which is the tell.
 3. **The story genuinely doesn't exist**.

Tell them apart by whether a plain story listing works: if listing succeeds but one story 404s, suspect permissions or a stale id; if everything 404s, fix `host` first.

- **429 Too Many Requests** — Tines rate-limits per token and per tenant. Nothing is misconfigured; retry with a smaller result limit.
- **TLS / certificate error** — a self-hosted tenant using a self-signed or internal-CA certificate; enable **Skip TLS verification** on the connector.